Final Project Report

Course Team Project - Deliverable #2

CE/CS/SE 3354 Software Engineering

SRS + SDD + SPMP (IEEE 1058) + Test Documentation (IEEE 829) + Final Evaluation

0. Document Control

Project Title	*UTD Student Life App*
Team Name / #	*Aakash, Parinita Chilukuri, Prakrit,Aamir Khan,Hector Lopez,William Turner, Shayaan- Team 9*
GitHub Repository URL	*github.com/AKdaAKA/F26_3354_Team09*
Report Date / Version	*[11/15/2026] - v[3]*

1. Task Delegation

Clearly describe delegation of tasks - who did what across the full project. Distribute tasks fairly among team members. If a member had no or poor contribution, specify this explicitly so each student can be graded fairly.

Team Member	Task(s) for Deliverable #1	Contribution Status
Aakash Bhagavathi	*Deliverable 1* *Team GitHub repository created and included. Feedback Addressed and Functional Requirements sections written.* *Deliverable 2* *Acted as project manager. Responsible for section 3 (schedule), Function Point cost, effort and pricing estimate. Finalised the report.*	*Full*
Prakrit Chauhan	*Completed section 4.4 Requirements traceability matrix Functional requirements based on events*	*Full*

Aamir Khan	*Requirements (create event, RSVP) Use Case Diagram 2 and its description reviewed*	*Full*
Hector Lopez	*Section 4.2: Non-functional requirements listing All operational constraints*	*Full*
Parinita Chilukuri	*Software Process Model (see section 3). Architectural Design – Client Server Diagram and rationale of section 5.3 Initial commit to repository with Readme (Task 1.4)*	*Full*
Shayaan Rahim	*Class Diagram (See section 5.2). Database Design inputs (PostgreSQL)*	*Full*
William Turner	*Use Case diagram (see section 4.3). Sequence Diagrams (see section 5.1) All 8 sequence diagrams have been created from the 8 use cases.*	*Full*

2. Deliverable #1 Content (Carried Forward)

Include everything already submitted in Course Team Project Deliverable #1, updated with any instructor feedback received since that submission. Reproduce the subsections below in this exact order - this is the same structure used in your Deliverable #1 report, so you can copy each section directly.

2.1 Task Delegation (Deliverable #1)

Team Member	Task(s) for Deliverable #1	Contribution Status
Aakash Bhagavathi	*Team GitHub repository was created and added to the report, and sections for*	*Full*

	Feedback Addressed and Functional Requirements were written in the report.	
Prakrit Chauhan	*Done with Section 4.4 requirements traceability matrix*	*Full*
Aamir Khan	*Event-related functional requirements (create event, RSVP) and review of Use Case Diagram 2 and its description*	*Full*
Hector Lopez	*Section 4.2: Listing all non-functional requirements regarding the operational constraints*	*Full*
Parinita	*Software Process Model (Section 3) and Architectural Design - Client Server Diagram & Rationale for Section 5.3 First Repository Commit With Readme (Task 1.4)*	*Full*
Shayaan	*Section 5.2 Class Diagram; database-related design inputs (PostgreSQL)*	*Full*
William Turner	*Section 4.3 Use Case diagram: created diagram & use case descriptions Section 5.1 Sequence Diagrams: created all 8 sequence diagrams from the 8 use cases*	*Full*

2.2 Feedback Addressed from Project Proposal

Feedback Received on Proposal	How We Addressed It in Deliverable #1
*[*Lokesh Balasubramaniam/In scope 3 and 4 solves the same problem. Please address it and come up with a new scope feature. Only 4 constraints mentioned instead of 5.*]*	The two scope items in points 3 and 4 were combined into one. A new scope item was added to replace this; allowing students to connect with each other and send direct messages. This was defined as a new scope item as it is not covered within the merged point. The new scope items were defined as the following: FR-7 – Double opt-in connection requests. FR-8 – Direct messages between connected students. A new constraint was also added. This was a technological constraint requiring the system to use a stable database that could cope with high usage periods.

2.3 Software Process Model

We will be using an Agile/Scrum-based model. Using two-week sprints, a prioritized product backlog, and also a sprint review at the end of each sprint.

Justification:

This model works better compared to other models primarily due to the fact that we have a small team and a target end date. We need a process that produces working software early and tolerates change, and Scrum fits that better than the alternatives for a few reasons. Our requirements are still shifting and Scrum allows us to reprioritize the backlog every sprint when one of these dependencies falls through, whereas Waterfall would lock requirements in before we know what is actually feasible. Our timeline is also short and has fixed checkpoints, and we have a small team already divided by speciality. Due to this, we are able to have sprint reviews which fit our work week timings. We also need a small group of students to test the app, as mentioned in the project proposal, and the Scrum's sprint reviews allow us to demo working features to testers and fold their feedback in before any deadlines, instead of finding usability issues towards the end.

2.4 Software Requirements Specification (SRS)

2.4.1 Functional Requirements

ID	Requirement	Rationale / Notes
FR-1	***Password shall be stored in salted hash-format for security*** ***(As a student, I would like to store my password in salted hash format (not in plain text) to make sure my account is protected if the database is compromised.)***	*It protects login credentials and personal information of user profiles from being misused by hackers.*
FR-2	***The app shall use UTD's login credentials to make it a closed-app within the UTD student body*** ***(As a student, I would like to register with my UTD email and receive a onetime code via it to make sure only registered UTD students may register.)***	*To limit registration only to UTD students and prevent unauthorized access to our app.*
FR-3	***The app shall have a search functionality that can be filtered on name, major and interest (profile features)*** ***(As a student, I would like to look for other students based on name, major and interests.)***	*To easily find other students with common interests.*
FR-4	***The app shall have a RSVP feature that notifies event organizers*** ***(As a student, I would like to be able to RSVP on an event and to see the number of people who have RSVP'ed, to prevent RSVPing to an event if it's already full.)***	*To let students be committed to attending events, and to give hosts ability to control the capacity of event.*

FR-5	***The app shall have a post-content reporting functionality*** ***(As a Student I want to be able to report posts that are abusive and let a moderator review over them and take them down)***	Inappropriate posts provide an unsafe environment for meeting other people especially since this is an online app.
FR-6	***The app shall have a profile page that can be customized with personal information (names, interest, major)*** ***(As a student, I want to create a profile with my name, major, and interests, so that other students can learn who I am.)***	Lets users sort through similar profiles of similar hobbies.
FR-7	***The app shall implement a double-opt in system for interactions between students*** ****As a student, I want to send a connection request to another student and have them accept or decline it, So that I only chat with people who agree to connect.)***	*The accept decline system helps protect students against unwanted requests.*
FR-8	***The app shall provide direct message capability between students*** ***(As a student, I want to send and receive direct messages with students I'm connected to, So that I can get to know them and make plans.)***	*Messaging helps extend short term comments into long lasting conversations for its users.*
FR-9	***The app shall have an event creation feature that lets students host campus gatherings*** ***(As a student, I would like to be able to create an event with a title, time, and capacity limit, so that other students can discover it and RSVP.)***	*To let students organize their own activities and give them control over the details of the events they host.*

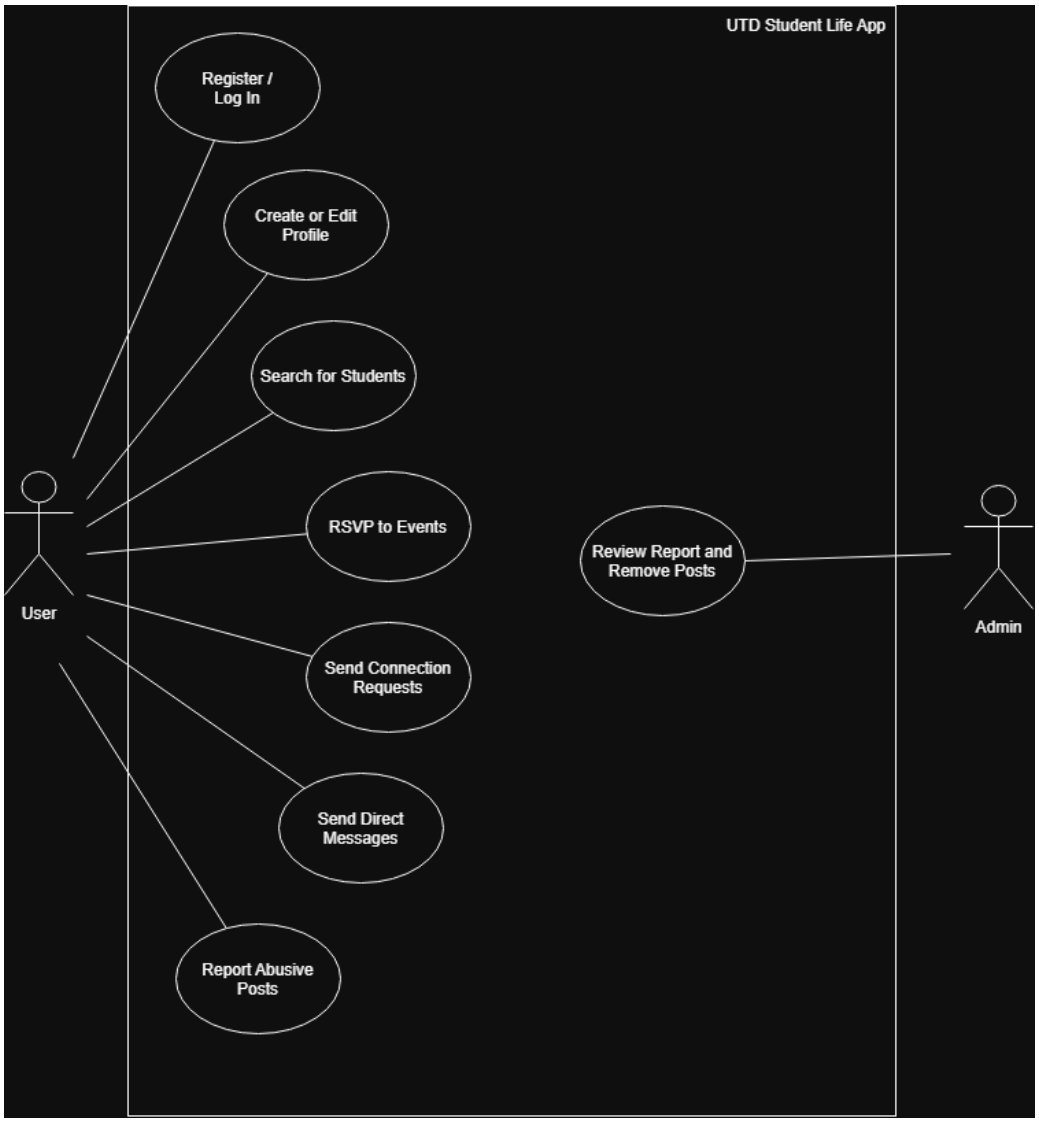
2.4.2 Non-Functional Requirements

NFR Type (Fig. 4.3 leaf)	**Category**	**Requirement Statement**
Performance	Product - Efficiency	*The app should not load the main campus feed in over 2 seconds at normal network speeds.* *The app shall not exceed 800 ms when pulling read request (loading comments, user profile)* *The app shall be able to support a minimum of 150 request per second during normal campus hours(8:00am - 3:00pm)* *Cloud server cpu usage shall not exceed 80% on normal load and usage* *The app should not use more than 250mb of ram on user's device on normal use*
Space	Product - Efficiency	*The app shall dynamically record wifi bandwidth strength and change compression ratios based on the wifi strength. Speeds around 512kb would result in data payload compressions at 100kb per payload. Beyond that, compression should remain constant at a limit of 1MB. App size should be less or equal to 50MB, and bandwidth usage to be 3MB/min.*
Reliability	Product - Dependability	*The app shall have 100% boot rate on first boot.* *The app should have a crash-free session rate of 99%* *The app shall have gracious failure measures rather than a full-app crash* *The central server should have an uptime of 99.9% during normal use cases*
Availability	Product - Dependability	*The app shall remain accessible and usable to users 24/7 (in exception of scheduled downtime/maintenance)* *App shall advise users of upcoming downtime 12 hours in advance* *Downtime shall be done in times where app usage is the lowest*
Security	Product - Dependability	*The data transmitted between the client to the central server shall be encrypted* *The app shall verify user permissions every request to the main server* *The app shall restrict any access to the database that stores users information*

Safety	Product - Dependability	*The app shall encrypt and privatize user reports* *Resolved user reports shall be purged within 90 days*
Usability	Product	*The user shall be able to go from the home page to post creation within 2 taps* *The user shall be able to undergo the first login procedure in a maximum of 3 minutes* *The app shall have consistent button placement (home, back, exit) throughout all pages*
Environmental	Organizational	*The app shall have support for new mobile OS (IOS 27 and Android 17.0)* *The app shall support older mobile OS versions down to iOS 16 and Android 12. This app is strictly mobile with no web version.*
Operational	Organizational	*The application shall monitor its central server cpu usage and advise administrators if it exceeds 80% for over 5 minutes* *The app shall have a clear flow for reports to be viewed by moderators within seconds of it being received*
Development	Organizational	*The system shall be built based on the client-server architectural pattern* *The system shall use native mobile frameworks like SwiftUI for the iOS client and native Android for the Android client.*
Interoperability	External	*The app will work with UTD's email system to provide single use verification codes for students to sign up.* *The app shall integrate third-party cloud storage such as AWS S3 if required* *The app shall be able to utilize platform-native push notifications (Android/IOS)*
Ethical	External	*The software shall not incorporate any means to backdoor user information for non-app related functions.* *The software shall not incorporate any type of device surveillance*

		The central server's database shall only be used for essential app services and not sold to third-parties.
Legislative - Privacy	External - Legislative	*The app shall comply with the TDPSA act* *The app shall comply with the FTC Act* *The app shall comply with the FERPA Act*
Legislative - Safety	External - Legislative	*The app shall support emergency data retention protocols in order to provide information to requested authorities if safety of students is deemed at risk*
Legislative - Security	External - Legislative	*The app will encrypt student data while in transmission (HTTPS/TLS) and at rest to ensure protection of student records complying with FERPA and the TDPSA.*

2.4.3 Use Case Diagram



Use Case Descriptions

Briefly describe each use case shown in the diagram (actor, goal, brief flow).

- Use Case 1 – Register / Log In (UC-1):

  Actor(s): Student.

  Goal: Create an account and securely access the app.

  Brief flow: The student enters their UTD email and password. The system sends a one-time verification code to the email. The student enters the code, and the system verifies it and creates the account. Passwords are stored securely using salted hashes.

- Use Case 2 – RSVP to Event (UC-2):

  Actor(s): Student.

  Goal: Join an event and see how many students are attending.

Brief flow: The student opens an event and selects RSVP. The system checks whether the event has available capacity. If space is available, the RSVP is saved and the attendee count is updated. Otherwise, the system rejects the RSVP.

- Use Case 3 – Report Abusive Post (UC-3):

  Actor(s): Student, Moderator.

  Goal: Keep the app safe by reporting inappropriate content.

  Brief flow: A student reports an abusive post. The system saves the report and makes it available to a moderator. The moderator reviews the post and can remove it if it violates the rules.

- Use Case 4 – Search for Students (UC-4):

  Actor(s): Student.

  Goal: Find other students with shared interests.

  Brief flow: The student enters a name, major, or interest into the search interface. The system searches student profiles and displays matching results. The student can select a profile to view its information.

- Use Case 5 (UC-5): Create/Edit Profile

  Actors: Student

  Goal: To create personal profile for other students to know about you.

  Process: Student navigates to their profile page, enters the details of their name, major and interests. The details are validated and the profile will be created. If any mandatory information is missing, the profile won't be updated and will display an error.

- Use Case 6 (UC-6): Send Connection Request

  Actors: Student

  Goal: To connect with another student.

  Process: Student A clicks 'Connect' on Student B's profile. The system will check if a request already exists and if not, store the request as pending. Student B will be notified, they will either accept or decline. If they accept a connection will be created.

- Use Case 7 (UC-7): Send Direct Message

  Actors: Student

  Goal: To message a student they are connected to.

  Process: Student A types a message and sends it to Student B. The system will check if they are connected. If they are, the message will be stored and sent to Student B. If they aren't, it'll be rejected.

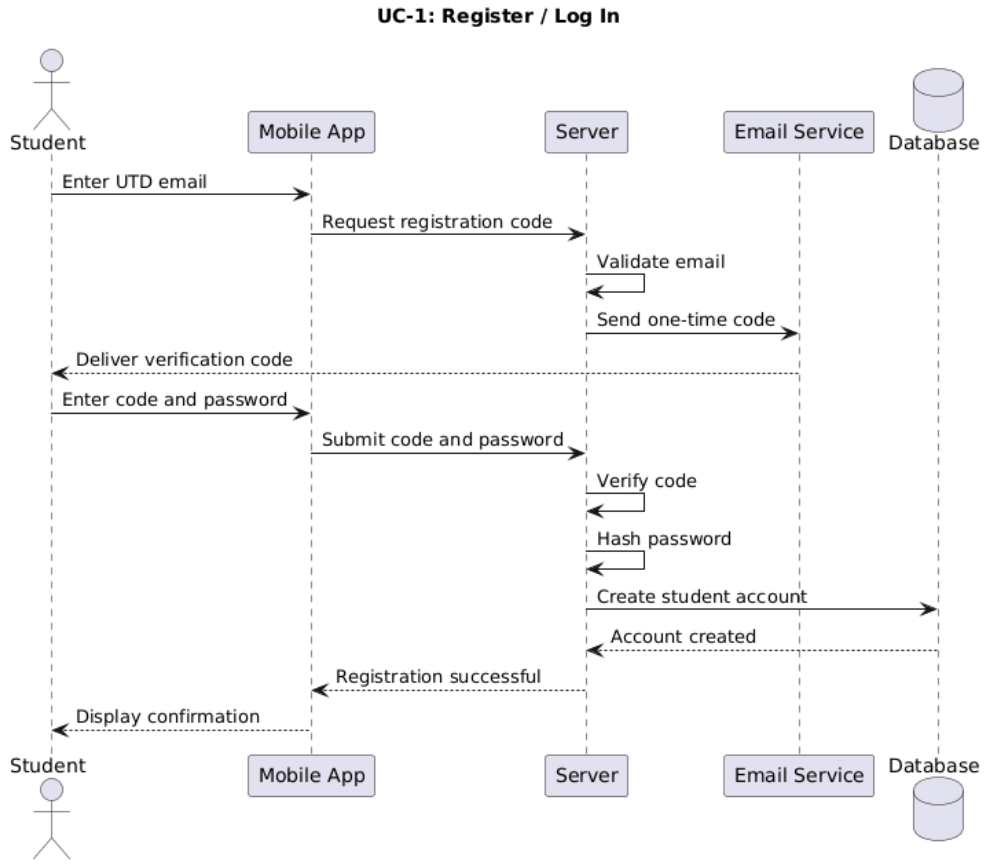
2.4.4 Requirements Traceability Matrix

Req ID	Requirement (Summary)	Source (Use Case)	Design Element (Class/Component)	Verification Method
FR-1	*Store credentials securely using hashing*	*[UC-1]*	*User, Auth Service*	*Inspection of database (to ensure there is no plaintext). Unit test of hashing.*
FR-2	*Sign up using UTD email and one-time code*	*[UC-1]*	*Student, Auth Service, OTP*	*Check whether a valid UTD email will be able to receive a code; a non-UTD email will get rejected, as will expired/invalid codes.*
FR-3	*Search for students using their name, major, or interests*	*[UC-4]*	*Student, Profile, Search Service*	*Ensure that a search finds relevant profiles and that no search results when blank.*
FR-4	*RSVP to an event and see the attendee count update in real time*	*[UC-2]*	*RSVP, Event, Event Service*	*Test RSVP, cancel, and count refresh. Test that RSVP is rejected when the event is full. Demo.*
FR-5	*Report abusive posts. A moderator reviews and removes them*	*[UC-3]*	*Report, Post, Moderator, Thread Service*	*Test report a post, moderator removes it, post disappears. Demo with a moderator account.*
FR-6	*Create a profile with name, major, and interests*	*[UC-5]*	*Student, Profile, Profile Service*	*Test valid and empty fields, edit and save. Inspect that the profile shows on other users' views.*
FR-7	*Send a connection request that the other student can accept or decline*	*[UC-6]*	*Connection Request, Student, Connection Service*	*Test send, accept, decline, and duplicate requests. Demo with two accounts.*
FR-8	*Send and receive direct messages with connected students only*	*[UC-7]*	*Message, Connection, Student, Messaging Service*	*Test that connected users can message, unconnected users are blocked. Inspect that*

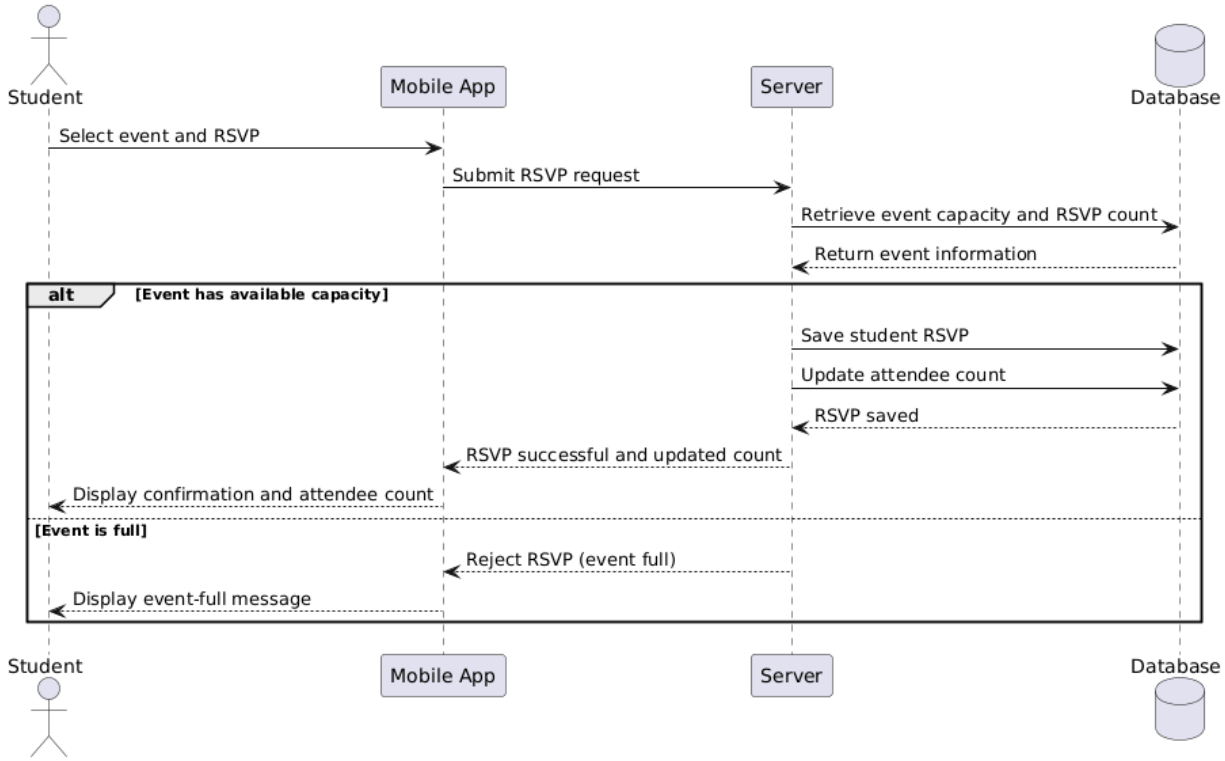
				messages are stored securely.

2.5 Software Design Description (SDD)

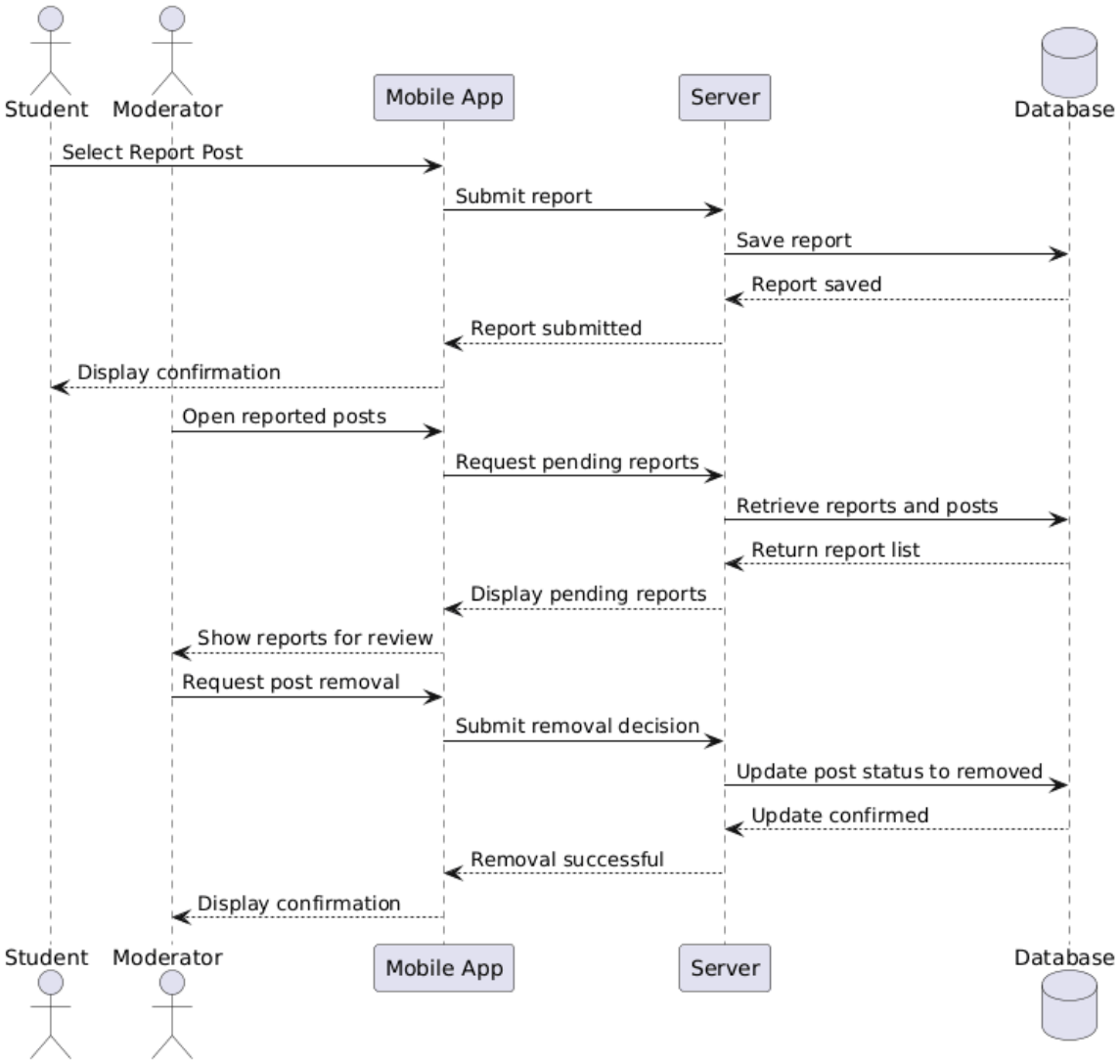
2.5.1 Sequence Diagrams

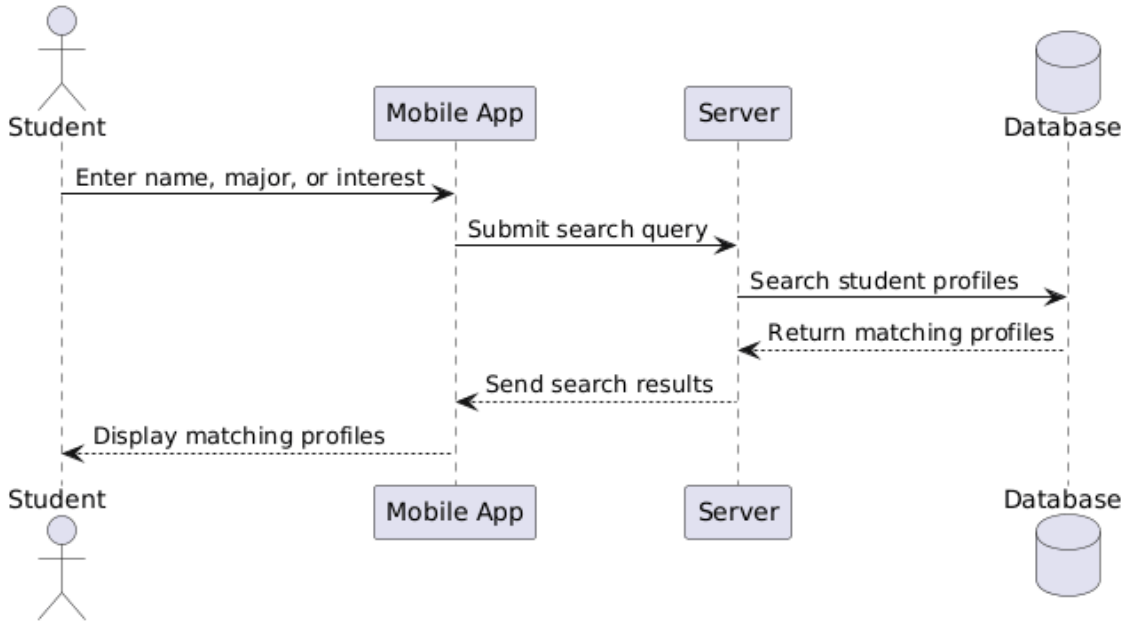


UC-2: RSVP to Event

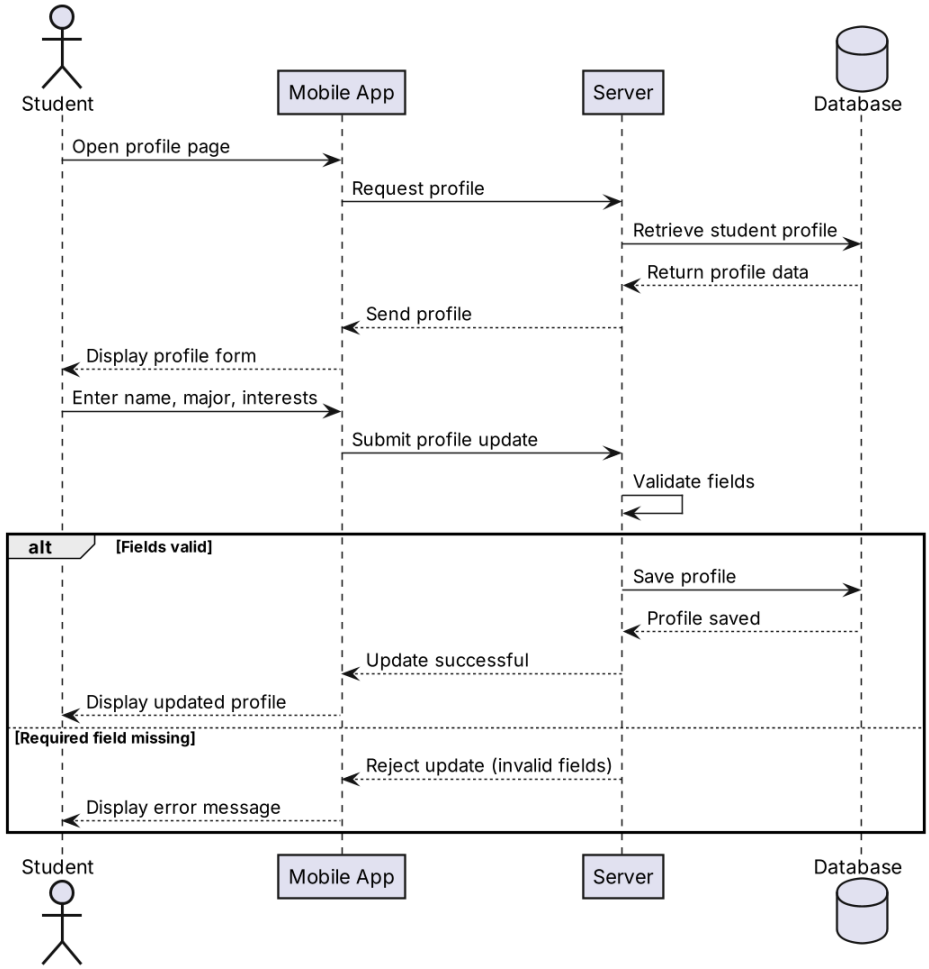


UC-3: Report Abusive Post

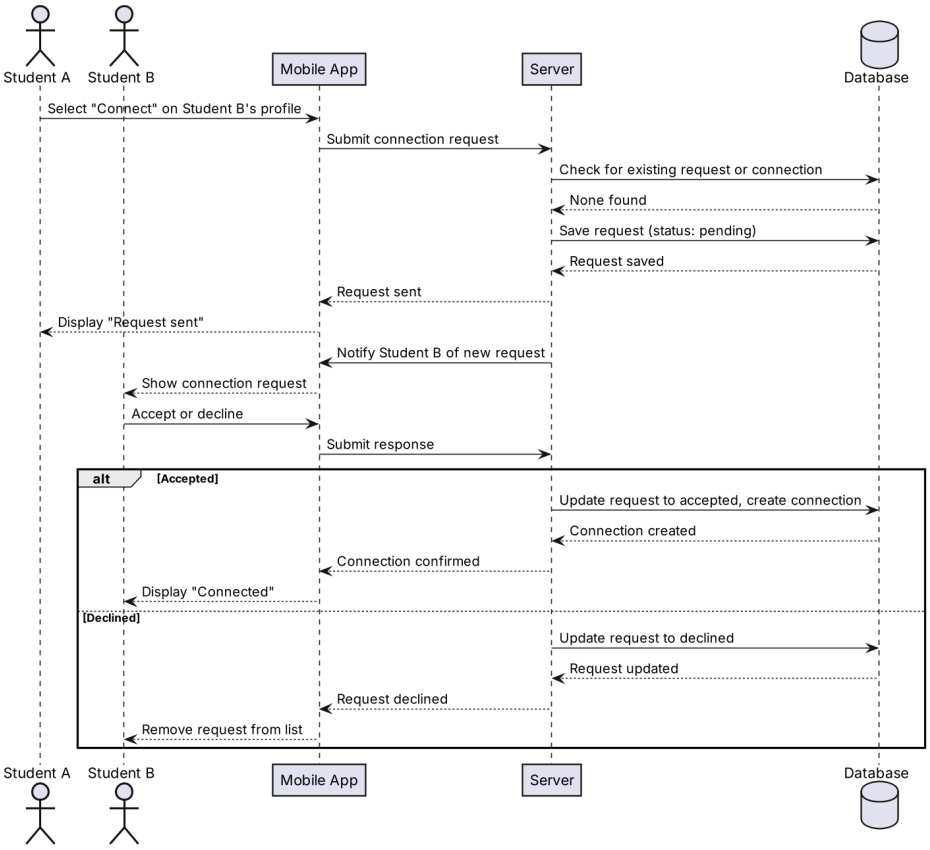




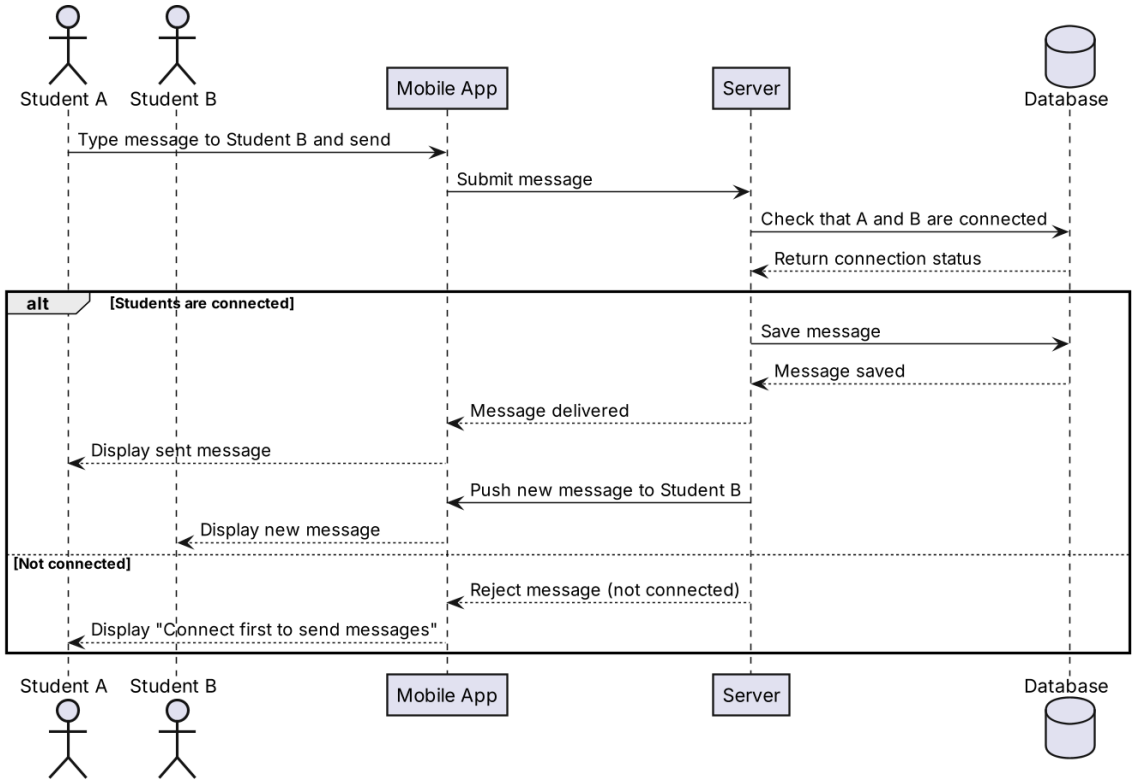
UC-5: Create or Edit Profile



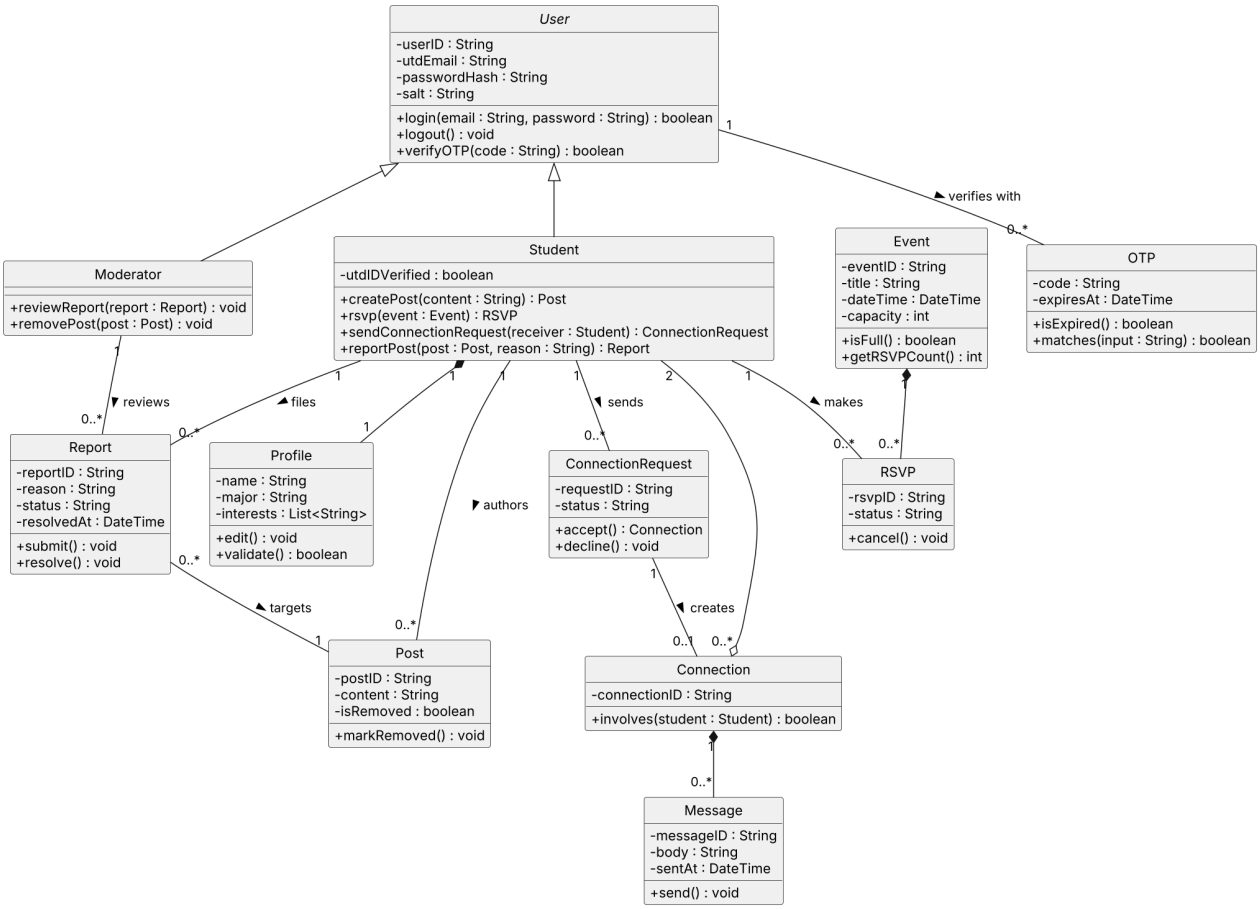
UC-6: Send Connection Request



UC-7: Send Direct Message



2.5.2 Class Diagram

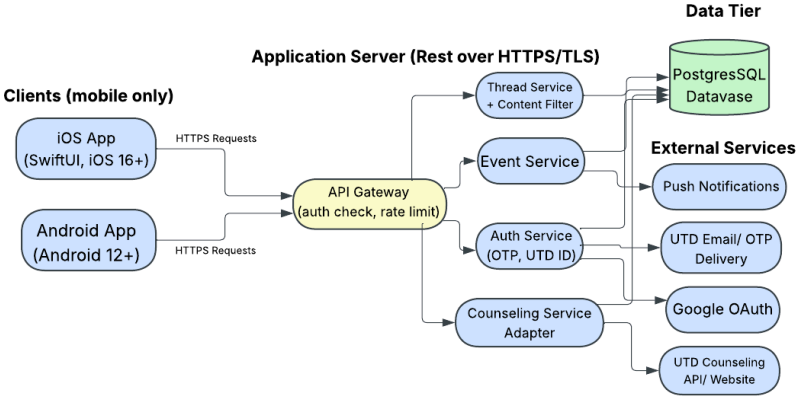


2.5.3 Architectural Design

Selected Pattern

Client-server architecture

Diagram



Justification

Client-server architecture fits our project because the app is mobile-only, and it has no web version, so every user runs the same client on their own phone.That is the standard client-server setup, with an app on the phone and the shared logic on a server. Another reason Client-server architecture works is because all students share the same data. Since events, RSVPs, threads, replies, and study groups have to be visible to everyone at once. So, a single central database behind one server keeps that data consistent, and an RSVP made on one phone would show up on every other phone. Also, the server enforces the rules like verifying a UTD email, one-time codes, checking sessions, and even removing the reported posts which can not depend on the phone. Keeping these on the server supports our security and safety requirements. Finally, it also fits out budget and timeline since one server and one database is simpler to build, test, and backup than a multi-service design, and it lets the frontend and backend developers work simultaneously.

2.6 GitHub Repository Setup

Task #	Task Description	Done By
1.1	Each team member should create a GitHub account if they do not already have one.	*All Team Members*
1.2	Create a GitHub repository named **F26_3354_TeamNumber**.	*Aakash*
1.3	Add all team members, and the TA/Grader, as collaborators. (TA/Grader GitHub ID will be provided once assigned.)	*Aakash*
1.4	Make the first commit to the repository: a **README file** with [Team Number and Team Name] as its content.	*Parinita*
1.5	Make another commit including a **pdf** file named "Project Scope." This file's content should be identical to your topic's description that your team chose from one of the predefined topics. (Section 4 of your Project Charter)	*Prakrit*
1.6	Include the **URL** of your team's project repository in your Deliverable #1 report.	William

3. Project Scheduling, Cost, Effort, and Pricing Estimation

IMPORTANT: *The items below are calculated based on the project you are designing (e.g., a hospital information system), NOT based on your team of students. You are estimating the effort to build the software product itself, as if by a professional engineering team, not the effort your class team spent on this course.*

3.1 Project Scheduling

Provide a start date, end date, and justification for your estimation. Also specify whether weekends are counted and how many working hours per day are assumed.

Project Start Date	*01/11/2027*
Project End Date	*06/01/2027*
Justification for Dates	*With an estimated function point effort of 25 person months, by having a professional team of 5 engineers working in parallel we can achieve this task in around 5 months which would be 100 working days. Starting work on Monday, January 11th 2027 (First full week of the Spring Semester) and working 100 days (Excluding MLK Day, 01/18/2027 and Memorial Day, 05/31/2027) means we will finish on Tuesday, June 1st 2027. This will give us time to submit to App Store and Google Play, along with doing a summer beta with a small number of students between June and July. The app will be live in time for Orientation and Welcome Week during the Fall 2027 semester where new students are most likely to be looking to connect.*
Weekends Counted?	*No. The team will work 5 days a week only. The weekends and the two federal holidays are not considered as work days.*

Working Hours / Day	*8 hrs/day/engineer. 1 man-month = 4 weeks 40 hours = 160 hrs.*

3.2 Cost, Effort, and Pricing Estimation

Choose ONE of the two alternative cost modeling techniques below and apply it consistently: ● Function Point (FP) Analysis

● Application Composition (e.g., object/component-based estimation)

Selected Technique

Function Point Analysis (IFPUG weights [1])

Calculation & Justification

1. Calculate Weighted Functions

Functions listed in the SRS (FR-1 to 9, UC-1 to 7), class diagram and architecture have been identified as being either an External Input (EI), External Output (EO), External

Inquiry (EQ), Internal Logical File (ILF) or External Interface File (EIF). They have also been given a complexity rating of Low, Average or High. Each function type has been given its IFPUG weight [1]. EI: 3/4/6, EO: 4/5/7, EQ: 3/4/6, ILF: 7/10/15, EIF: 5/7/10.

EI

• Account Register (A)

• OTP Code (L)

• Login (L)

• Create/Edit Profile (A)

• Create Event (A)

• RSVP/Cancle RSVP (A)

• Create Post/Reply (A)

• Report Post (L)

• Moderator Approve/Delete Post (A)

• Send Connection request (L)

• accept/decline request (L)

• send direct message (A)

5xLow = 15 7xAverage = 28

EI = 43

EO

• Verify OTP (Email) (L)

• View Event Attendees/RSVP Status (A)

• Push Notifications (RSVP, Request, Message) (A)

• Moderator Report queue (A)

• admin CPU alert (L)

• downtime advisory (L)

3xLow = 12 3xAverage = 15

EO = 27

EQ

• Search Student (Name, Major, Interest) (H)

• View Profile (L)

• View Campus Feed (A)

• View Event Info (L)

• View Conversation (A)

• View Pending Requests (L)
3xLow = 9 2xAverage = 8 1xHigh = 6
EQ = 23

ILF
• User (A)
• Credentials (A)
• Profile (A)
• Event (A)
• RSVP (L)
• Post/Thread (A)
• Report (L)
• ConnectionRequest/Connection (L)
• Message (A)
3xLow = 21 5xAverage = 50
ILF = 71

EIF
• UTD Email (OTP) (L)
• APNs/FCM (Push Notification) (L)
• AWS S3 (Cloud Storage) (L)
3xLow = 15
EIF = 15

UFP (Unadjusted Function Points) = 43+27+23+71+15= 179

2. Value Adjustment Factor

Each of the 14 General System Characteristics have been scored between 1-5, based on their influence. 0 = No Influence, 5 = Essential.

• 1. Data communications All data sent/received between client/server using HTTPS: 4
• 2. Distributed Data Processing: 2
• 3. Performance Feed takes 2 seconds to load. Read takes 800ms to load.: 4
• 4. High usage configuration 80% CPU Limit: 3
• 5. High transaction rate 150 requests/sec: 4

• 6. On-line data entry: 5

• 7. Efficiency 2 tap post. 3min onboarding: 4

• 8. Online update (RSVP, Post, Message): 4

• 9. Complex Processing (Mostly CRUD, Moderation): 2

• 10. Reusable: 2

• 11. Easy to install (App Stores): 3

• 12. Easy to operate (Monitoring, Alerts): 3

• 13. Multiple sites (iOS & Android Clients): 3

• 14. Facilitate change: 3

Total Degree of Influence (TDI)= 46

VAF = 0.65 + (TDI x 0.01) = 0.65 + (0.46) = 1.11

FP Adjusted = UFP x VAF = 179 x 1.11 = 198.69 ~ 199 FP

3. Effort

We assumed our productivity rate would be 8 FP per person-month. A productivity rate higher than 10 FP per staff month is a good rate [2]. A rate below 7 FP is not so good [2]. However QSM show their median business project took 21.85 person-months to produce 160 FP. This gives a productivity rate of approximately 7.3 FP per PM [3]. As we had to develop two native clients (SwiftUI & Android) and a backend we decided to go with a mid-point value.

• E = FP / Productivity Rate = 199 / 8 = 24.9 ~ 25 Person-Months. 25 x 160h = 4000 hours.

• 25 Person-Months / 5 Engineers = 5 Months. (Approx 100 working days) (See Section 3.1).

4. Cost & Price

Used the 2025 median salaries for Software Developers, QA Analysts and Testers from BLS [4].

• Development Labour Cost = 4000h x $64.44 = $257,760.00

• Project Cost = $260,337.60 (Personnel) + $8,145.00 (Hardware) + $224.00 (Software) = $268,706.60 (See 3.3).

• Client Price (UTD) = Cost x 1.20 (20% Profit/Contingency Margin) = $322,447.92

3.3 Estimated Cost of Hardware, Software, and Personnel

Estimated Hardware Cost	*$8,145.00* *5x 13" MacBook Air M4 ($999 [6]) (Xcode needs to be used for macOS)* *$4,995* *Test devices (2 iPhones, 2 Android devices) (~$600)* *$2,400* *Server and PostgreSQL Database (~$150 per month)* *5 months (~$750)*
Estimated Software Cost	*$224.00* *Apple Developer Program (per year) ($99 [5])* *Google Play Developer Account Setup ($25 [5])* *GitHub Team (4 per user per month x 5 users x 5 months) ($100)* *Xcode, Android Studio, PostgreSQL, JUnit ($0) (Free/Open source)*
Estimated Personnel Cost	*$260,337.60* *5 people (1 iOS Developer, 1 Android Developer, 1 Backend Developer, 1 Full Stack/Security Developer, 1 QA Engineer).* *4000 hours of coding ($64.44 per hour [4]) = $257,760.00* *Post installation Training (40 hours x $64.44) = $2,577.60 (Training studentlife staff and moderators on using the moderation queue and admin alerts plus a user guide).*

4. Test Plan

4.1 Test Plan Description

Describe the test plan for testing a minimum of one unit of your software (e.g., a class or

method).

Unit Under Test:

Event.addRSVP(String studentID), Event.isFull(), Event.getRSVPCount(), RSVP.cancel()

These are methods directly from the Event and RSVP classes shown in our class diagram (Section 2.5.2). They satisfy the requirement FR-4 / UC-2 (Add RSVP to event. If event is full cancel RSVP).

Scope:

Only testing the business logic associated with RSVPs on the back-end. Mobile UI, REST API Layer and PostgreSQL database are not being tested. RSVPs are stored in memory inside the Event.

Approach:

This testing method uses a black box testing technique. Equivalence partitioning and boundary value analysis were used to find different test cases for this unit. This includes: Adding an RSVP when there is space in the event, Adding an RSVP when the event has maximum capacity, Cancelling an RSVP to create space, Adding duplicates and Adding an RSVP when event has 0 capacity (invalid). The test is automated using JUnit 4.13.2 and Java 21. The test was carried out using the command line so that it can easily be run again after every change (regression testing).

Objective(s):

(1) Add RSVP and increase attendee count. (2) Cancel RSVP and do not increase count. (3) Cancellation creates space. (4) Student can only add one RSVP. (5) Events cannot be created with capacity less than 1. All 5 test cases need to pass with no failures.

4.2 Unit Code & Automated Test Results

Provide the code for one unit (e.g., a method) of your software, and the automated testing tool used (e.g., JUnit for Java) along with its output.

Event.java — F26_3354_Team09

Event.java | EventRSVPTest.java

src › edu › utd › studentlife › Event.java

```java
package edu.utd.studentlife;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/** Campus event that students can RSVP to (FR-4, FR-9, UC-2). */
public class Event {
    private final String eventID;
    private final String title;
    private final LocalDateTime dateTime;
    private final int capacity;
    private final List<RSVP> rsvps = new ArrayList<>();

    public Event(String eventID, String title, LocalDateTime dateTime, int capacity) {
        if (capacity < 1) {
            throw new IllegalArgumentException("capacity must be at least 1");
        }
        this.eventID = eventID;
        this.title = title;
        this.dateTime = dateTime;
        this.capacity = capacity;
    }

    /** Number of active (non-cancelled) RSVPs. */
    public int getRSVPCount() {
        int count = 0;
        for (RSVP r : rsvps) {
            if (r.isActive()) count++;
        }
        return count;
    }

    public boolean isFull() {
        return getRSVPCount() >= capacity;
    }

    /**
     * UNIT UNDER TEST. Saves an RSVP for the student if the event has space.
     * Rejects the RSVP if the event is full or the student already has an active RSVP.
     */
    public RSVP addRSVP(String studentID) throws EventFullException {
        for (RSVP r : rsvps) {
            if (r.isActive() && r.getStudentID().equals(studentID)) {
                throw new IllegalStateException("Student " + studentID + " already RSVP'd");
            }
        }
        if (isFull()) {
            throw new EventFullException(eventID);
        }
        RSVP rsvp = new RSVP(eventID + "-" + (rsvps.size() + 1), studentID);
        rsvps.add(rsvp);
        return rsvp;
    }

    public String getEventID() { return eventID; }
    public String getTitle() { return title; }
    public LocalDateTime getDateTime() { return dateTime; }
    public int getCapacity() { return capacity; }
}
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java

EventRSVPTest.java — F26_3354_Team09

EventRSVPTest.java | Event.java

test › edu › utd › studentlife › EventRSVPTest.java

```java
package edu.utd.studentlife;

import static org.junit.Assert.*;

import java.time.LocalDateTime;
import org.junit.Before;
import org.junit.Test;

public class EventRSVPTest {
    private Event event;

    @Before
    public void setUp() {
        event = new Event("E100", "Board Game Night", LocalDateTime.of(2026, 11, 20, 19, 0), 2);
    }

    /** TC-1: RSVP when space is available is saved and the count updates. */
    @Test
    public void tc1_rsvpWithSpaceIsSaved() throws Exception {
        RSVP r = event.addRSVP("S001");
        assertEquals(RSVP.CONFIRMED, r.getStatus());
        assertEquals(1, event.getRSVPCount());
        assertFalse(event.isFull());
    }

    /** TC-2: RSVP to a full event is rejected and the count does not change. */
    @Test
    public void tc2_rsvpToFullEventIsRejected() throws Exception {
        event.addRSVP("S001");
        event.addRSVP("S002");
        assertTrue(event.isFull());
        try {
            event.addRSVP("S003");
            fail("Expected EventFullException");
        } catch (EventFullException expected) {
            // expected
        }
        assertEquals(2, event.getRSVPCount());
    }

    /** TC-3: Cancelling an RSVP frees a seat so another student can join. */
    @Test
    public void tc3_cancelFreesSeat() throws Exception {
        RSVP first = event.addRSVP("S001");
        event.addRSVP("S002");
        first.cancel();
        assertEquals(1, event.getRSVPCount());
        assertFalse(event.isFull());
        event.addRSVP("S003");
        assertEquals(2, event.getRSVPCount());
    }

    /** TC-4: The same student cannot RSVP twice to the same event. */
    @Test(expected = IllegalStateException.class)
    public void tc4_duplicateRsvpIsRejected() throws Exception {
        event.addRSVP("S001");
        event.addRSVP("S001");
    }

    /** TC-5: An event cannot be created with a capacity below 1. */
    @Test(expected = IllegalArgumentException.class)
    public void tc5_invalidCapacityIsRejected() {
        new Event("E101", "Empty Event", LocalDateTime.of(2026, 11, 21, 18, 0), 0);
    }
}
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java



4.3 Test Cases & Results

Clearly define the test case(s) used for testing and the results obtained.

Test ID	Input / Setup	Expected Result	Actual Result
TC-1	*Create an event with capacity of 2 and add an RSVP from S001.*	*1 person (Capacity not filled)*	*Confirmed - 1 person (Capacity not filled) Pass*

TC-2	*Add another RSVP from S002. Attempt to add RSVP from S003.*	*2 people An exception is raised.*	*2 people Pass*
TC-3	*There are 2 people in the event (S001 and S002). S001 cancels their RSVP. S003 adds an RSVP.*	*Cancelled 1 person (Capacity not filled)*	*Confirmed - S003 2 people (Capacity filled) Pass*
TC-4	*S001 adds an RSVP. Attempt to add duplicate RSVP from S001.*	*Exception should be thrown. Duplicate not allowed*	*Exception thrown. Pass*
TC-5	*Create event with capacity = 0.*	*Exception should be thrown. Event not created.*	*Exception thrown. Pass*

5. Comparison of Your Work with Similar Designs

This requires thorough search in the field of your project domain. Please cite any references you make.

Similar System / Reference	**How It Compares to Our Design**	**Citation**
UT Dallas official mobile app	*This is similar as it is a free UTD app which contains information about events happening on campus, push notifications and a directory of student organisations (350+). The difference is that our app focuses on scheduling, coursework and official university content. There are no ways to search for other students, send connection requests or send direct messages. Our app focuses on connecting students (FR-3, FR-7, FR-8) and the creation of events by students (FR-9). We have the benefit of being an official university endorsed app with current users.*	*[7] – see References*
Fizz (campus social app)	*This is similar as it is an exclusive campus community which requires students to verify using their school email (Utd email, one time code FR-2) and removes any posts which go against the rules (Like our FR-5). This is different as it focuses on anonymous posts whereas we use real profiles (Name, Major and Interests) to create real friendships. We also add the ability to RSVP to events with capacity limits. Our design is smaller and does not have a marketplace, meaning there is*	*[8] – see References*

	less opportunity to cyberbully anonymously.	
Bumble For Friends (BFF)	*This is similar as it helps you find new friends based on interests, double opt-in (Both people must agree to chat with each other FR-7) and helps organise group meetups. However it is different as it is open to anyone nearby whereas we only allow verified Utd students. It also uses swipe and paid upgrades whereas we use major/interest search. Our exclusive campus community makes it a safer environment for students but has less users.*	*[9] – see References*
EngageUTD	*This is similar as it is a Utd platform which allows students to find and sign up to events. However it only allows students to find volunteering opportunities from community partners and keeps track of service hours. Students do not have the option to create their own events or connect with each other. Our RSVP and event creation features (FR-4, FR-9) add onto this idea allowing any student hosted event.*	*[10] – see References*

6. Conclusion

Evaluate your work. Describe any changes you needed to make (if any), whether things deviated from what you originally planned, and give justification for such changes.

Overall, we have met the aims of this project. We have created a full requirements document, design, timeline, budget and a tested RSVP unit. As we had a team of 7 members and tight deadlines for this project, using Scrum worked well. For each item in our deliverables, we used sprints to resolve any issues based on feedback.

We did have some changes to our initial proposal. (1) Scope: TA mentioned that there were 2 in scope points that were solving the same issue. We also had very few constraints of only 4. We merged the duplicate scope and added new features: double-opt-in connection requests (FR-7) and direct messages (FR-8). We also added a 5 th constraint about having a stable database during high periods. (2) Security: Whilst initially our aim was to make sure that everything was end-to-end encrypted, based on our own constraints it would be very difficult to achieve this in just 3 months. As a result we changed our security requirements to use HTTPS/TLS encryption during transit and encryption at rest. We also use salted password hashes (FR-1). This still meets the FERPA and TDPSA. (3) Counseling: We don't include any scheduling functionality for counselors. Instead we use a Counseling Service Adapter which connects to the current counselling site at UTD. We can't access internal APIs. (4) Platform: We have developed this as a mobile only application (SwiftUI and native android). We have not created a web version to keep our client server architecture simple.

From our Function Point calculations, we can see that a professional version would be much larger than this (199FP = ~25 person months = $268,706.60). Our biggest risk remains making

sure students use the application. If we were to move forward, we would create the rest of our services and expand our automated testing to cover the authentication and messaging services. We would also complete our summer beta with student testers as outlined in our proposal.

7. References

Include properly cited references in IEEE numbered referencing format - references should be numbered, and those numbers cited throughout your report.

[1] International Function Point Users Group, "Function point counting practices: Reference guide, IFPUG CPM version 4.1," Univ. of Kansas EECS. [Online]. Available: people.eecs.ku.edu/~saiedian/Teaching/811/Papers/Function-Points/fpa-ref.pdf

[2] C. Jones, "Interview with Capers Jones on measuring for agile adoption," InfoQ. [Online]. Available: www.infoq.com/articles/Jones-measuring-agile-adoption

[3] Quantitative Software Management, "Function point analysis: The 'average' function point project," QSM. [Online]. Available: www.qsm.com/httpdocs/pdf.php?nid=584

[4] U.S. Bureau of Labor Statistics, "Software developers, quality assurance analysts, and testers," Occupational Outlook Handbook, U.S. Dept. of Labor. [Online]. Available: www.bls.gov/ooh/computer-and-information-technology/software-developers.htm. Accessed: Nov. 10, 2026.

[5] SplitMetrics, "Google Play and Apple App Store fees," SplitMetrics Blog, Aug. 8, 2025. [Online]. Available: splitmetrics.com/blog/google-play-apple-app-store-fees/

[6] "Apple unveils M4 MacBook Air in new color and cheaper starting price," 9to5Mac, Mar. 5, 2025. [Online]. Available: 9to5mac.com/2025/03/05/apple-unveils-m4-macbook-air-in-sky-blue-color-and-cheaper-starting-price/

[7] The University of Texas at Dallas, "UTD mobile application," UT Dallas. [Online]. Available: www.utdallas.edu/utdmobile-application/. Accessed: Nov. 10, 2026.

[8] "Fizz," Lightspeed Venture Partners. [Online]. Available: lsvp.com/company/fizz/. Accessed: Nov. 10, 2026.

[9] Bumble Inc., "Bumble Inc. introduces 'Bumble For Friends' friendship-finding app," Business Wire, Jul. 26, 2023. [Online]. Available: www.businesswire.com/news/home/20230726935186/en/

[10] The University of Texas at Dallas, "EngageUTD," Office of Student Volunteerism. [Online]. Available: volunteer.utdallas.edu. Accessed: Nov. 10, 2026.

GitHub & eLearning Submission Checklist

At least ONE team member must commit/submit everything listed below to BOTH GitHub and eLearning. Missing items result in a 10-point deduction per location (GitHub requirement and eLearning requirement are graded separately).

Required Item	**Committed to GitHub?**	**Submitted to eLearning?**
Deliverable #2 report	*Yes*	*Yes*
Unit test code for a sample unit	*Yes*	*Yes*
Implementation code (if implemented)	*Yes*	*Yes*
Updated Readme.md	*Yes*	*Yes*
Presentation slides	*Yes*	*Yes*
Presentation recording (max 15 min, eLearning only)	*Yes*	*Yes*